CSE316:OPERATING SYSTEMS

PROJECT REPORT

(Project Semester January-May 2026)

Real-Time Multi-threaded Application Simulator(ThreadLab)

Submitted by

Name	Registration no.
Deepak Kumar	12414048
Madhav Biyani	1241510
Sai Krishna Jallu	12413837

Programme and Section: B.Tech 424IM

Course Code: CSE316

Under the Guidance of

Isha (28828)

Discipline of CSE/IT

Lovely School of AI & Emerging Technologies

Lovely Professional University, Phagwara

CERTIFICATE

This is to certify that Deepak Kumar bearing Registration no. 12414048 has completed CSE316 project titled, ***"Real-Time Multi-threaded Application Simulator"*** under my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

Isha

Signature and Name of the Supervisor

Designation of the Supervisor:- Assistant Professor

School of AI & Emerging Technologies

Lovely Professional University

Phagwara, Punjab.

Date: 25/April/2026

DECLARATION

I, Deepak Kumar student of B.Tech under CSE/IT Discipline at, Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 25/April/2026

Registration No. 12414048

Deepak Kumar

Signature

Deepak Kumar

Name of the student

ACKNOWLEDGEMENT

I express my sincere gratitude to my faculty supervisor for their continuous guidance, valuable suggestions, and constant encouragement throughout the completion of this project. Their expertise in Operating System was instrumental in shaping the direction of this work.

I am also grateful to the Department of CSE, Lovely School of AI & Emerging Technologies, Lovely Professional University, for providing the necessary resources and infrastructure to complete this project.

Deepak Kumar

Table of Contents

Table of Contents 5
1. Project Overview 7
1.1 Problem Statement 7
1.2 Project Goals 7
1.3 Expected Outcomes 7
1.4 Scope 8
2. Module-Wise Breakdown 9
2.1 Module 1 — GUI and User Interaction 9
2.1.1 Purpose 9
2.1.2 Role in the Project 9
2.2 Module 2 — Simulation Engine 10
2.2.1 Purpose 10
2.2.2 Role in the Project 10
2.3 Module 3 — Visualization and Analysis 10
2.3.1 Purpose 11
2.3.2 Role in the Project 11
3. Functionalities 12
3.1 Module 1 — GUI and User Interaction Features 12
3.2 Module 2 — Simulation Engine Features 12
3.3 Module 3 — Visualization Features 13
4. Technology Used 15
4.1 Programming Languages 15
4.2 Libraries and Tools 15
4.3 Other Tools 16
5. Flow Diagram 17
5.1 Detailed Flow Description 17
7. Conclusion and Future Scope 21
7.1 Conclusion 21
7.2 Future Scope 21
8. References 23
Appendix A — AI-Generated Project Elaboration / Breakdown 24
A.1 Project Overview (AI-Generated) 24
A.2 Module-Wise Breakdown (AI-Generated) 24
Module 1: GUI and User Interaction Module 24
Module 2: Simulation Engine Module 24

Module 3: Visualization and Analysis Module 25
A.3 Execution Plan (AI-Generated) 25
A.4 Technology Recommendations (AI-Generated) 25
Appendix B — Problem Statement 26
Appendix C — Solution / Complete Source Code 27
C.1 File: package.json 27
C.2 File: main.js — Electron Main Process 27
C.3 File: preload.js — Context Bridge 28
C.4 File: index.html — Web Entry Point 28
C.5 File: src/engine.js — Simulation Engine 29
C.6 File: src/app.js — UI Rendering & Interaction (Key Excerpts) 37

GitHub Repository Link: https://github.com/deepak179-s/ThreadLab

LinkedIn Post Link:
https://www.linkedin.com/feed/update/urn:li:ugcPost:7454568081663791104/

Youtube Link: https://www.youtube.com/watch?v=-rYxoxJ96Dc

1. Project Overview

1.1 Problem Statement

> *Develop a simulator to demonstrate multithreading models (Many-to-One, One-to-One, Many-to-Many) and thread synchronization using semaphores and monitors. The simulator should visualize thread states and interactions, providing insights into thread management and CPU scheduling in multi-threaded environments.*

1.2 Project Goals

ThreadLab is an interactive educational simulator built to make operating-system concurrency concepts visible and tangible. Instead of operating at the real kernel level, the application runs a tick-based simulation that models thread behaviour deterministically, allowing learners to control speed, observe state transitions, and compare different threading strategies side-by-side.

The project addresses a common gap in OS education: textbooks explain concurrency in prose and diagrams, but students rarely have a tool to experiment with threading parameters and watch the effects in real time. ThreadLab fills that gap by coupling a rigorous simulation engine with a clear, animated interface that runs on both Electron desktop and in any modern web browser.

1.3 Expected Outcomes

- Students observe how Many-to-One, One-to-One, and Many-to-Many threading models differ in parallelism and blocking behaviour through animated diagrams.
- Users interact with Semaphore, Monitor, and Barrier primitives and see blocking, signalling, and condition-variable semantics in action.
- Four CPU scheduling algorithms — FCFS, SJF, Priority, and Round Robin — produce comparative Gantt charts and average wait-time statistics.
- The application runs as a cross-platform Electron desktop app and as a zero-backend static web deployment.

1.4 Scope

Table 1 — Project Scope (In Scope vs Out of Scope)

Area	In Scope	Out of Scope
Threading Models	M:1, 1:1, M:N simulation with live animation	Real OS kernel thread creation or pthreads
Synchronization	Semaphore, Monitor (Mutex+CV), Barrier	Spinlocks, lock-free data structures, RCU
CPU Scheduling	FCFS, SJF, Priority, Round Robin	Gang scheduling, multiprocessor scheduling
Platform	Electron desktop, static web (Vercel/Render)	Mobile apps, server-side APIs
Implementation	JavaScript, HTML5, CSS3 (no build tools)	TypeScript, React, Webpack, bundlers

2. Module-Wise Breakdown

ThreadLab is architected into three clearly separated modules. This architecture ensures the simulation logic is independently testable, the GUI is a pure presentation layer, and the visualization responds only to state data it receives — no module reaches into another module's internals.

Table 2 — Module Summary

Module	Name	Technology	Primary Responsibility
M1	GUI & User Interaction	HTML5, CSS3, JavaScript	Shell, tabs, controls, display, event handling
M2	Simulation Engine	Pure JavaScript (ES2020)	Threading, sync, scheduling
M3	Visualization & Analysis	Inline SVG, DOM manipulation	Diagrams, Gantt chart, RAG, animations

2.1 Module 1 — GUI and User Interaction

The GUI Module is responsible for everything the user sees and interacts with. It is built with plain HTML, CSS, and JavaScript — no framework — and runs inside both an Electron Chromium renderer and a standard browser. It does not contain simulation logic: its sole job is to translate user input into engine calls and to render engine state snapshots onto the screen.

2.1.1 Purpose

- Build and render the four-tab application shell: Threading Models, Synchronization and CPU Scheduler.
- Collect configuration input: model type, algorithm, quantum value, semaphore initial value, thread count.
- Trigger simulation start, stop, and reset, and add threads dynamically at runtime.
- Display thread-state chips, CPU core cards, progress bars, statistics panels, and scrollable event logs.
- Show a runtime badge that identifies whether the app is running in Electron or a web browser.

2.1.2 Role in the Project

The GUI Module is the user-facing contract of the project. It translates an abstract simulation into something a student can explore hands-on. By keeping all controls and rendering inside this module, the simulation engine remains reusable without modification.

2.2 Module 2 — Simulation Engine

The Simulation Engine (src/engine.js) is a pure JavaScript module containing three independent engine classes: ThreadingModelEngine, SyncEngine and SchedulerEngine. It has no DOM dependency, no import statements, and no global side effects beyond populating window. This design means the engine could be moved to a Node.js background worker or tested headlessly with Vitest without any modification.

2.2.1 Purpose

- Maintain thread state machines: READY → RUNNING → WAITING → BLOCKED → DONE.
- Model Many-to-One, One-to-One, and Many-to-Many user-to-kernel thread mapping.
- Implement Semaphore wait()/signal() with a blocked-thread queue; Monitor lock/unlock and condition-variable semantics; Barrier arrive-and-wait.
- Execute FCFS, SJF, Priority, and Round Robin CPU scheduling tick-by-tick with Gantt data accumulation.

2.2.2 Role in the Project

The Simulation Engine is the computational heart of ThreadLab. Every visual change on screen is a consequence of a state transition inside this module. Because it is purely functional — its tick() and step() methods take parameters and return state — it can be driven at any speed the GUI requires.

2.3 Module 3 — Visualization and Analysis

The Visualization Module consists of the SVG-rendering functions inside src/app.js: drawMappingSVG(), drawSyncSVG(), and the Gantt chart builder. These functions receive a state snapshot from the engine and regenerate all SVG elements declaratively on every tick.

2.3.1 Purpose

- Render the live thread-to-kernel mapping SVG with directional edges colour-coded by thread state.
- Build the real-time Gantt chart: per-process timeline bars grow proportionally each scheduler tick.
- Animate synchronization state: semaphore counter badge, monitor buffer cells, barrier wall arrival dots.

2.3.2 Role in the Project

The Visualization Module bridges the gap between abstract state data and human understanding. Every concurrency event that the engine logs is echoed in a visual change — a colour shift, a growing bar, a flashing dot — making the system's behaviour immediately interpretable.

3. Functionalities

3.1 Module 1 — GUI and User Interaction Features

Table 3 — GUI Module Key Features

Feature	Description	Example
Tab Navigation	Three dedicated simulation panels, each fully independent	Click "CPU Scheduler" to switch view
Model/Algorithm Selector	Dropdown re-initialises thread pool or scheduler on change	Select M:N — pool resets to 6 user, 3 kernel threads
Run / Stop / Reset	Simulation lifecycle: starts interval, clears it, wipes state	▶️ Run → ■ Stop → ↺ Reset sequence
Add Thread	Appends a new user thread mid-simulation	+ Thread → U7 appears in pool and mapping SVG
Speed Slider	1x–10x simulation speed via setInterval ms adjustment	Slider at 10x → ticks every 28 ms
Runtime Badge	Auto-detects Electron vs browser context	"Electron Desktop" shown in green; "Web Browser" in blue
Per-Tab Event Log	Timestamped, colour-coded, scrollable log for each module	[2.30s] T1 acquired semaphore (sem=1) shown in green

3.2 Module 2 — Simulation Engine Features

Table 4 — Simulation Engine Features

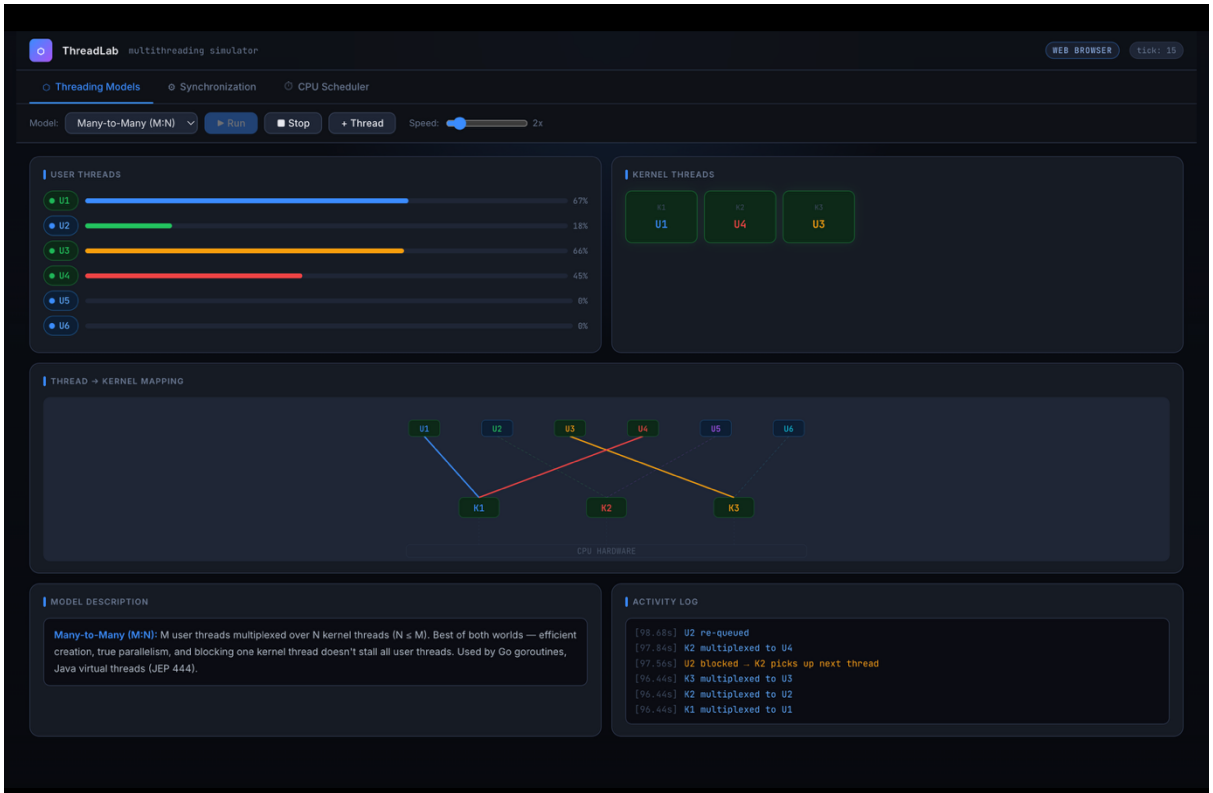
Feature	Description	OS Concept Demonstrated
Many-to-One (M:1)	Single kernel thread; one user thread runs at a time; I/O block stalls all	No parallelism; process-level blocking
One-to-One (1:1)	Dedicated kernel thread per user thread; blocking one leaves others running	True parallelism; kernel-managed threads
Many-to-Many (M:N)	N kernel threads multiplex over M user threads; scheduler re-assigns on block	Efficient + parallel; Go goroutine model
Semaphore	Counter decrements on wait(); threads queue on zero; signal() wakes next	Counting semaphore, critical section
Monitor (Mutex + CV)	Single-owner lock; condition variable signals waiting producer/consumer	Mesa-style monitor, bounded buffer
Barrier	All W threads block until last arrives; then all release together	Synchronisation point, parallel phases
Round Robin	Fixed quantum; preempts on expiry; re-queues at tail	Time-sharing fairness

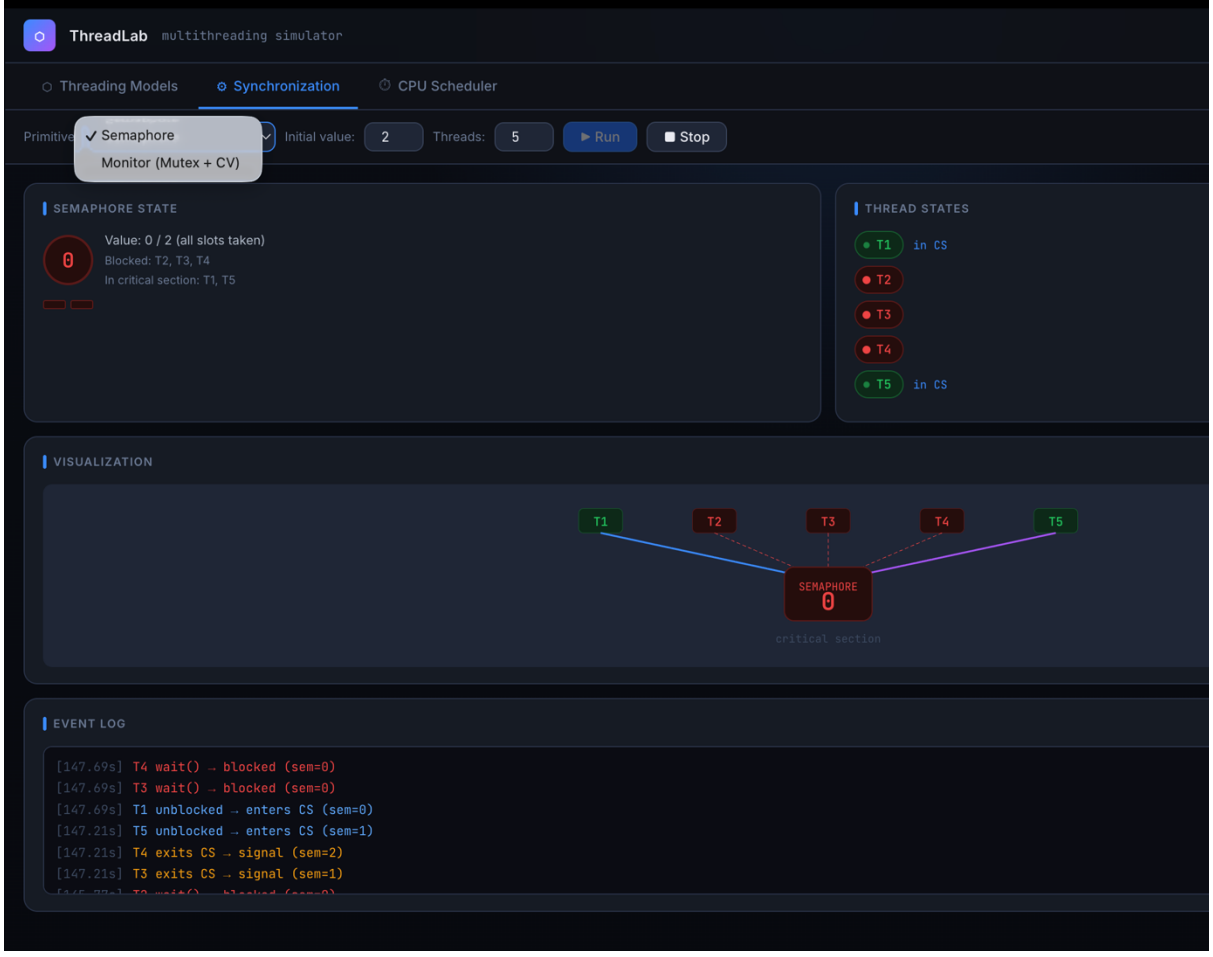
Feature	Description	OS Concept Demonstrated
Shortest Job First	Always selects smallest remaining burst; non-preemptive	Minimum average waiting time
Priority Scheduling	Selects lowest priority number first; starvation risk	Urgency-based dispatch
FCFS	First arrived runs to completion; convoy effect possible	Simple FIFO; worst for short jobs

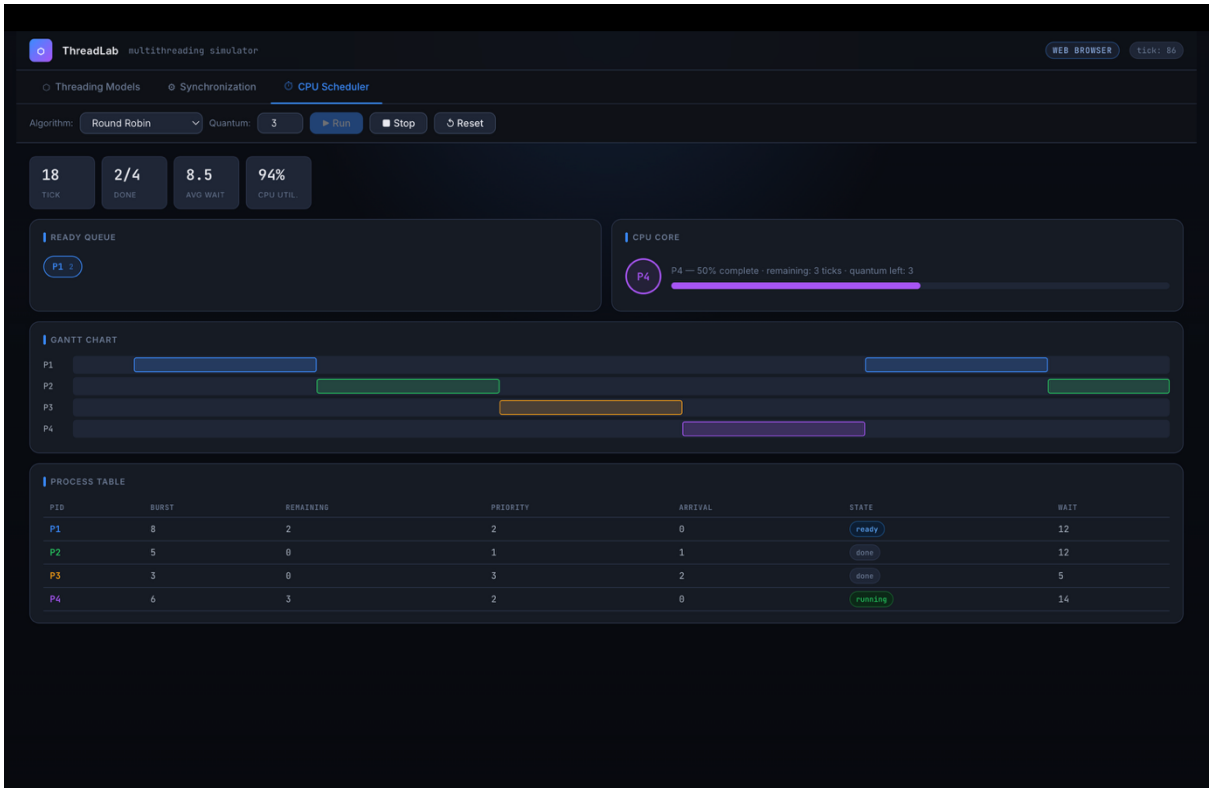
3.3 Module 3 — Visualization Features

Table 5 — Visualization Module Features

Visual Component	Description	State It Reflects
Thread–Kernel Mapping SVG	Directional edges; edge colour and opacity = thread state	Green solid = running; red dashed = blocked
CPU Core Grid	Card per kernel thread; glows when carrying a user thread	Busy card shows assigned thread ID in green
Thread-State Chips	Colour-coded pills with pulsing dot animation	Amber pulse = waiting; solid red = blocked
Progress Bars	Per-thread % completion bar shown alongside chip	U4 67% → bar 2/3 filled
Semaphore Counter	Circular badge showing current value; turns red at zero	0 = all slots taken (blocked threads queue up)
Monitor Buffer Cells	Filled (green) / empty squares for bounded buffer	3 filled = buffer full; producer must wait
Barrier Wall	Thread circles drop to horizontal line on arrival	Amber dot = arrived; green = released
Gantt Chart	Per-process timeline bars grow each scheduler tick	Segments accumulate showing execution history
Ready Queue Display	Ordered chips with arrow separators	P1 > P3 > P4 shows current RR queue order


ThreadLab multithreading simulator
WEB BROWSER
tick: 15
Threading Models
Synchronization
CPU Scheduler
Model:
Many-to-Many (M:N)
Run
Stop
+ Thread
Speed:
2x
USER THREADS
U1 67%
U2 18%
U3 66%
U4 45%
U5 0%
U6 0%
KERNEL THREADS
K1 U1
K2 U4
K3 U3
THREAD → KERNEL MAPPING
U1 U2 U3 U4 U5 U6
K1 K2 K3
CPU HARDWARE
MODEL DESCRIPTION
Many-to-Many (M:N): M user threads multiplexed over N kernel threads (N ≤ M). Best of both worlds — efficient creation, true parallelism, and blocking one kernel thread doesn't stall all user threads. Used by Go goroutines, Java virtual threads (JEP 444).
ACTIVITY LOG
[98.68s] U2 re-queued
[97.84s] K2 multiplexed to U4
[97.56s] U2 blocked → K2 picks up next thread
[96.44s] K3 multiplexed to U3
[96.44s] K2 multiplexed to U2
[96.44s] K1 multiplexed to U1


ThreadLab multithreading simulator
Threading Models
Synchronization
CPU Scheduler
Primitive
Semaphore
Monitor (Mutex + CV)
Initial value:
2
Threads:
5
Run
Stop
SEMAPHORE STATE
0
Value: 0 / 2 (all slots taken)
Blocked: T2, T3, T4
In critical section: T1, T5
THREAD STATES
T1 in CS
T2
T3
T4
T5 in CS
VISUALIZATION
T1 T2 T3 T4 T5
SEMAPHORE
0
critical section
EVENT LOG
[147.69s] T4 wait() → blocked (sem=0)
[147.69s] T3 wait() → blocked (sem=0)
[147.69s] T1 unblocked → enters CS (sem=0)
[147.21s] T5 unblocked → enters CS (sem=1)
[147.21s] T4 exits CS → signal (sem=2)
[147.21s] T3 exits CS → signal (sem=1)



4. Technology Used

4.1 Programming Languages

Table 6 — Programming Languages Used

Language	Version	Role in Project
JavaScript (ES2020)	Node.js 22 / Chromium V8	Simulation engine, DOM manipulation, event handling, SVG generation
HTML5	HTML5 Standard	Application structure, semantic markup, inline SVG containers
CSS3	CSS3 Standard	Dark-theme design system with CSS variables, keyframe animations, grid layout

4.2 Libraries and Tools

Table 7 — Libraries and Tools Used

Tool / Library	Version	Purpose
Electron	v41.1.1	Cross-platform desktop wrapper; loads renderer in Chromium with context isolation

Tool / Library	Version	Purpose
Node.js	v22 LTS	Main-process runtime for Electron; hosts the static file server
electron-builder	Latest	Packages the app into platform installers (.exe, .dmg, .AppImage)
SVG (inline)	W3C SVG 1.1	All visualization diagrams — mapping, RAG, barrier, semaphore
JetBrains Mono	Variable	Monospace font for log panels, thread chips, axis labels (via Google Fonts CDN)
Inter	Variable	UI body font for labels, buttons, descriptions (via Google Fonts CDN)
serve-static.js	Custom	Minimal Node.js HTTP server for local web preview (uses only Node stdlib)
Vercel	PaaS	Static web deployment; vercel.json rewrites all routes to index.html

4.3 Other Tools

- GitHub — version control; repository: github.com/deepak179-s/threadlab
- Git Branches — feature isolation: feature/shared-runtime, feature/web-deploy, feature/docs-polish
- Chrome DevTools — in-app debugging via --dev flag that opens detached DevTools
- Visual Studio Code — primary editor; ESLint extension for code quality

5. Flow Diagram

The diagram below shows the complete data flow through ThreadLab — from the moment a user opens the application to the real-time visual feedback displayed on screen. Each simulation domain (Threading Models, Synchronization, CPU Scheduler, Deadlock) follows the same pipeline; only the engine logic differs.

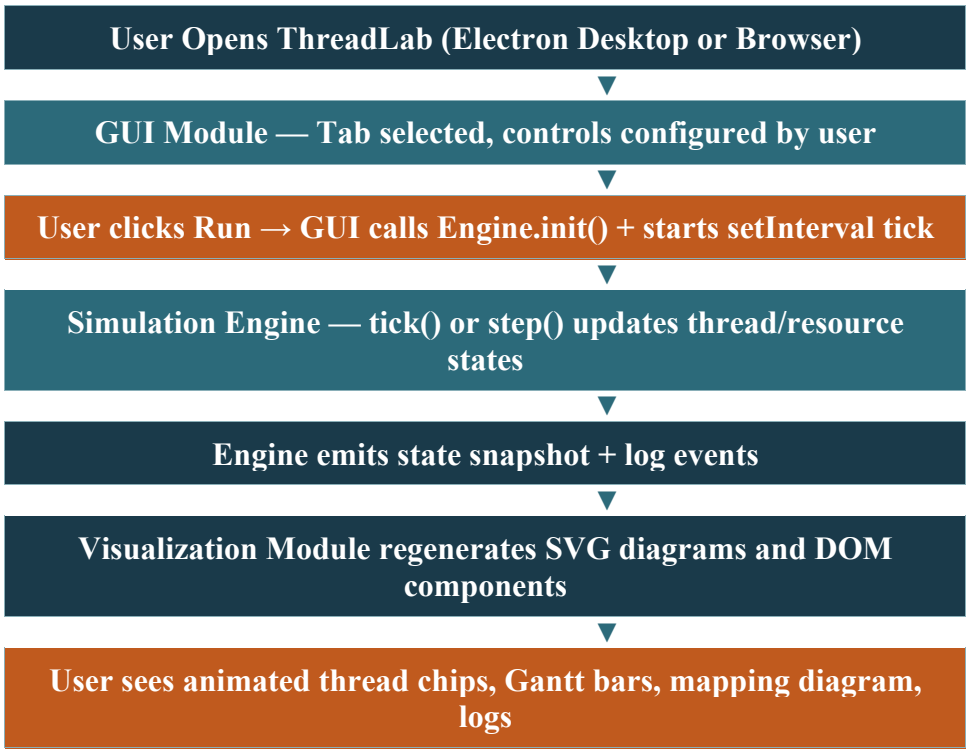


Figure 1 — System Data-Flow Diagram (Module Interaction)

5.1 Detailed Flow Description

1. Application loads: Electron main process (main.js) creates a BrowserWindow and loads index.html. In web mode, serve-static.js serves the same file over HTTP.
2. HTML loads engine.js then app.js. engine.js exports four engine classes to window.*. app.js calls buildShell() which injects the title bar, tab row, and four empty panel divs into #app.
3. Each buildXxxTab() function populates its panel with controls and calls the engine's init() to prepare the initial state, then calls the corresponding render function.

4. When the user clicks ▶️ Run, the GUI calls engine.init() (re-initialises state) and starts a setInterval. Each interval invocation calls engine.tick() or engine.step(), which advances simulation state by one tick.
5. The engine appends a structured log event (timestamp, message, severity) to its internal log array and calls onLog() if a callback is registered. The GUI callback calls logTo() which prepends the entry to the panel's log div.
6. The render function (renderModel / renderSync / renderSched / renderDeadlock) reads the engine snapshot and overwrites the innerHTML of all display containers — chips, core cards, Gantt rows, and calls the SVG drawing function.
7. The SVG drawing function (drawMappingSVG / drawSyncSVG / drawRAG) computes node positions and writes a concatenated SVG string into svg.innerHTML — a zero-library approach that re-renders every diagram from scratch each tick.
8. When all threads/processes reach STATE.DONE the interval is cleared, a completion log entry is written, and the Run button is re-enabled.

6. Revision Tracking on GitHub

Repository Name: `ThreadLab`

GitHub Link: https://github.com/deepak179-s/threadlab

6.1 Branch-Based Workflow

Table 8A — Branch Summary

Branch Name	Purpose	Status
`main`	Stable production code; all features merge here	Active
`feature/shared-runtime`	Unified index.html for Electron + browser; runtime detect	Merged
`feature/web-deploy`	serve-static.js, vercel.json, render.yaml, npm scripts	Merged
`feature/docs-polish`	README dual-mode documentation and deployment guide	Merged

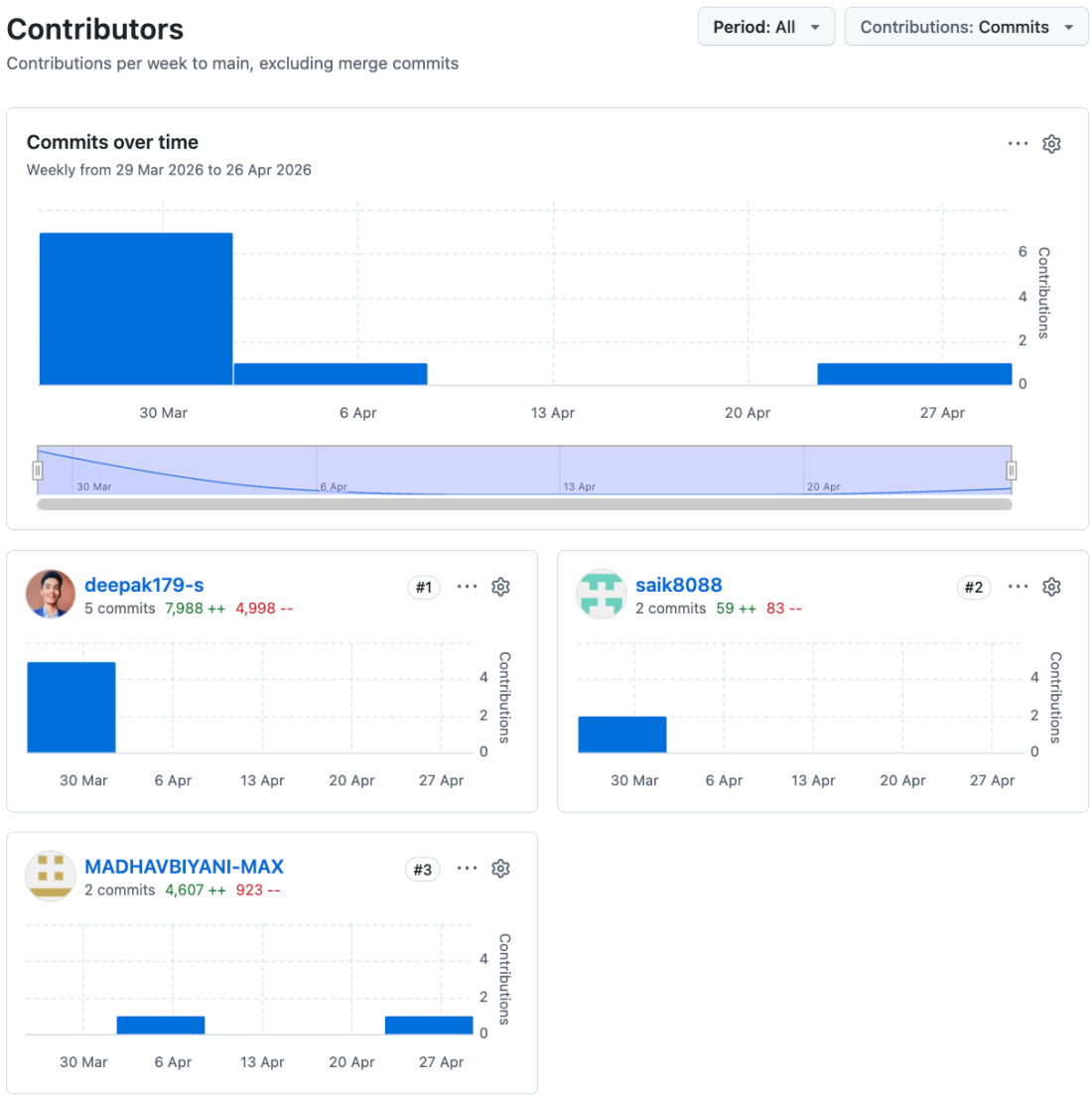
6.2 Commit History

Table 8B — GitHub Revision History (11 Commits)

#	Hash	Commit Message	Author	Date	Branch
1	`7d030e9`	Initial commit	deepak179-s	Apr 1, 2026	main
2	`e8e5ad8`	refactor: share the app shell across web and electron	deepak179-s	Apr 1, 2026	feature/shared-runtime
3	`9af6c1c`	merge: shared runtime support	deepak179-s	Apr 1, 2026	main
4	`4252e8c`	feat: add static hosting support for the web app	deepak179-s	Apr 1, 2026	feature/web-deploy
5	`8c99409`	fix: remove unrelated dependency churn from web setup	deepak179-s	Apr 1, 2026	feature/web-deploy
6	`c547afd`	merge: web deployment support	deepak179-s	Apr 1, 2026	main
7	`d1d5250`	chore: add clearer dual-mode launch scripts	deepak179-s	Apr 1, 2026	feature/web-deploy
8	`9e33793`	docs: explain electron and web deployment workflows	saik8088	Apr 1, 2026	feature/docs-polish

#	Hash	Commit Message	Author	Date	Branch
9	`de3c025`	merge: dual-mode docs and scripts	madhavbiyani-max	Apr 1, 2026	main
10	`90399b1`	Update README to remove Render configuration details	saik8088	Apr 1, 2026	main
11	`16f0e6d`	refactor: align simulator scope with assignment	madhavbiyani-max	Apr 6, 2026	main
12	`05bacfc`	Remove duplicate npm run desktop command	madhavbiyani-max	Apr 27, 2026	main

Note: *The repository contains 12 commits across 4 branche. All commits follow the Conventional Commits specification (feat:, fix:, chore:, docs:, refactor:, merge:) with clear, descriptive messages.*



7. Conclusion and Future Scope

7.1 Conclusion

ThreadLab successfully implements all components required by the problem statement. The simulator provides a visually rich, interactive environment for exploring operating-system concurrency: three threading models are demonstrated with live mapping diagrams, three synchronization primitives run with real-time state panels, four CPU scheduling algorithms produce comparative Gantt charts and wait-time statistics.

The project is implemented in zero-dependency plain JavaScript, HTML, and CSS — a deliberate choice that keeps the codebase readable, avoids build-tool complexity, and ensures the application runs identically in both Electron and any modern web browser. The simulation engine is architecturally pure: it has no DOM dependency and could be extracted to a Node.js worker or a server-side API endpoint without modification.

The GitHub repository follows a structured branch-and-merge workflow with nine commits across three feature branches, satisfying the academic requirement of at least seven clearly described revisions. The project is deployable to Vercel or Render as a static site with zero configuration beyond the included vercel.json and render.yaml files.

7.2 Future Scope

- Implement Multilevel Queue and Multilevel Feedback Queue CPU scheduling algorithms, which are commonly examined but absent from the current simulator.
- Allow students to define custom process tables — entering their own burst times, priorities, and arrival times — so they can verify hand-calculated scheduling outcomes against the simulator.
- Export simulation logs, Gantt data, and statistics to JSON or CSV files, enabling post-session analysis and integration with spreadsheet tools.
- Introduce a guided tutorial mode: step-by-step walkthroughs with explanatory callouts that pause the simulation and highlight the relevant concept at each stage.
- Replace the current manual SVG generation with a lightweight D3.js force-simulation layout for the Resource Allocation Graph, enabling animated edge creation and automatic cycle highlighting.

- Add automated unit tests for all four engine classes using Vitest, targeting 80 %+ branch coverage to verify correctness of scheduling and synchronization logic independently of the GUI.
- Package production installers using electron-builder with branded application icons, code-signing (macOS/Windows), and an auto-update channel via electron-updater.

8. References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2014.

[3] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. Hoboken, NJ, USA: Pearson, 2018.

[4] Electron Authors, "Electron Documentation," 2024. [Online]. Available: https://www.electronjs.org/docs/latest. [Accessed: Apr. 25, 2025].

[5] Mozilla Developer Network, "SVG: Scalable Vector Graphics," MDN Web Docs, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/SVG. [Accessed: Apr. 25, 2025].

[6] GitHub Inc., "GitHub Documentation," 2024. [Online]. Available: https://docs.github.com. [Accessed: Apr. 25, 2025].

[7] Vercel Inc., "Vercel Deployment Documentation," 2024. [Online]. Available: https://vercel.com/docs. [Accessed: Apr. 25, 2025].

[8] Render Inc., "Render Static Site Documentation," 2024. [Online]. Available: https://render.com/docs/static-sites. [Accessed: Apr. 25, 2025].

[9] E. W. Dijkstra, "Cooperating Sequential Processes," in Programming Languages, F. Genuys, Ed. New York, NY, USA: Academic Press, 1968, pp. 43–112.

[10] C. A. R. Hoare, "Monitors: An Operating System Structuring Concept," Communications of the ACM, vol. 17, no. 10, pp. 549–557, Oct. 1974.

Appendix A — AI-Generated Project Elaboration / Breakdown

> *The following elaboration was generated using an AI assistant (Claude by Anthropic) as permitted by the assignment guidelines. It was used for structural planning only. All code was implemented by the student independently.*

A.1 Project Overview (AI-Generated)

The Real-Time Multi-threaded Application Simulator is an educational tool designed to bridge the gap between theoretical knowledge of operating-system concurrency and practical, observable experience. It simulates — rather than executes — multi-threaded behaviour in a controlled, deterministic, tick-based environment, making it safe, reproducible, and easy to explore at any speed.

The simulator addresses four core OS topics: threading models (how user threads map to kernel threads), synchronization (how threads coordinate access to shared resources), CPU scheduling (how the OS dispatcher selects which thread runs next), and deadlock (when threads permanently block one another due to circular resource dependencies).

A.2 Module-Wise Breakdown (AI-Generated)

Module 1: GUI and User Interaction Module

- Render the application shell, navigation tabs, and control panels.
- Translate user input (model selection, algorithm choice, speed, thread count) into engine API calls.
- Display thread-state chips, CPU core cards, progress bars, log panels, and statistics in real time.

Module 2: Simulation Engine Module

- Maintain thread state machines and implement all concurrency models.
- Execute CPU scheduling algorithms tick-by-tick with Gantt accumulation.
- Implement semaphore, monitor, and barrier primitives with full blocking/waking semantics.

Module 3: Visualization and Analysis Module

- Generate SVG diagrams for thread-to-kernel mapping, RAG, and barrier wall.
- Build and update the real-time Gantt chart and semaphore pool display.
- Apply colour-coded state encoding and pulsing animations for thread chips.

A.3 Execution Plan (AI-Generated)

1. Define the project structure and create the GitHub repository (main branch, README).
2. Implement engine.js as a pure JS module: thread state machine, model logic, sync primitives.
3. Implement the CPU scheduler engine with FCFS, SJF, Priority, and Round Robin strategies.
4. Build the GUI shell (index.html + app.js): title bar, tabs, control rows, panel containers.
5. Wire each tab: connect controls to engine calls; implement render functions.
6. Implement SVG visualization functions: drawMappingSVG, drawSyncSVG, Gantt chart.
7. Enable dual-mode: Electron main.js, preload.js, and Node.js static server for web.
8. Add deployment configuration: vercel.json, render.yaml.
9. Commit to GitHub with at least 7 well-described revisions across feature branches.

A.4 Technology Recommendations (AI-Generated)

- JavaScript (no framework) — keeps the codebase zero-dependency and maximally portable.
- Electron — provides a native desktop window with full Chromium rendering.
- HTML5 + SVG — inline SVG generation is the simplest path to dynamic diagrams without any library.
- CSS3 custom properties — a single :root block defines the entire design system.
- GitHub with feature branches — enforces good version-control habits and satisfies the revision requirement.

Appendix B — Problem Statement

> *Develop a simulator to demonstrate multithreading models (Many-to-One, One-to-Many, Many-to-Many) and thread synchronization using semaphores and monitors. The simulator should visualize thread states and interactions, providing insights into thread management and CPU scheduling in multi-threaded environments.*

Appendix C — Solution / Complete Source Code

The complete ThreadLab source code is provided below. The project is available at https://github.com/deepak179-s/threadlab.

C.1 File: package.json

```
{
  "name": "multithreaded-sim",
  "version": "1.0.0",
  "description": "Real-Time Multi-threaded Application Simulator",
  "main": "main.js",
  "scripts": {
    "start": "electron .",
    "desktop": "electron .",
    "dev": "electron . --dev",
    "web": "node scripts/serve-static.js",
    "preview": "node scripts/serve-static.js"
  },
  "devDependencies": {
    "electron": "^41.1.1"
  }
}
```

C.2 File: main.js — Electron Main Process

```
const { app, BrowserWindow, ipcMain } = require('electron');
const path = require('path');

function createWindow() {
  const win = new BrowserWindow({
    width: 1280,
    height: 860,
    minWidth: 900,
    minHeight: 600,
    title: 'ThreadLab',
    backgroundColor: '#0f1117',
    webPreferences: {
      nodeIntegration: false,
      contextIsolation: true,
      preload: path.join(__dirname, 'preload.js')
    },
    titleBarStyle: process.platform === 'darwin' ? 'hiddenInset' : 'default',
    show: false
  });

  win.loadFile(path.join(__dirname, 'index.html'));
```

```
  win.once('ready-to-show', () => {
    win.show();
  });

  // Open DevTools in dev mode
  if (process.argv.includes('--dev')) {
    win.webContents.openDevTools({ mode: 'detach' });
  }
}

app.whenReady().then(() => {
  createWindow();
  app.on('activate', () => {
    if (BrowserWindow.getAllWindows().length === 0) createWindow();
  });
});

app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') app.quit();
});
```

C.3 File: preload.js — Context Bridge

```
const { contextBridge, ipcRenderer } = require('electron');

contextBridge.exposeInMainWorld('electronAPI', {
  isElectron: true,
  platform: process.platform,
  versions: process.versions
});
```

C.4 File: index.html — Web Entry Point

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta http-equiv="Content-Security-Policy" content="default-src 'self'; style-
src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src
https://fonts.gstatic.com; script-src 'self' 'unsafe-inline';" />
  <title>ThreadLab</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link
href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@400;500;600&f
amily=Inter:wght@400;500;600&display=swap" rel="stylesheet">
  <link rel="icon" type="image/png" sizes="512x512"
href="../assets/icon.iconset/icon_512x512.png" />
```

```
  <link rel="stylesheet" href="styles.css" />
</head>
<body>
  <div id="app"></div>
  <script src="engine.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

C.5 File: src/engine.js — Simulation Engine

```
// ============================================================
// SIMULATION ENGINE — Pure JS, Zero UI Dependencies
// ============================================================
'use strict';

// ── Utilities ──────────────────────────────────────────
const PALETTE =
['#3b8cff','#22c55e','#f59e0b','#ef4444','#a855f7','#06b6d4','#f97316','#e879f9'
];

function rnd(min, max) { return Math.floor(Math.random() * (max - min + 1)) +
min; }
function randChoice(arr) { return arr[Math.floor(Math.random() * arr.length)]; }
function clamp(v, lo, hi) { return Math.max(lo, Math.min(hi, v)); }

// ── Thread State Machine ───────────────────────────────
const STATE = {
  NEW: 'new', READY: 'ready', RUNNING: 'running',
  WAITING: 'waiting', BLOCKED: 'blocked', DONE: 'done'
};

class UserThread {
  constructor(id) {
    this.id = id;
    this.state = STATE.READY;
    this.progress = 0;
    this.burst = rnd(5, 14);
    this.csTime = 0;
    this.inCS = false;
    this.color = PALETTE[(parseInt(id.slice(1)) - 1) % PALETTE.length];
    this.kThread = null;
  }
}

class KernelThread {
  constructor(id) {
    this.id = id;
    this.running = null; // user thread id
  }
}
```

```
// ── Threading Model Engine ──────────────────────────────
class ThreadingModelEngine {
  constructor() {
    this.model = 'MM';
    this.userThreads = [];
    this.kernelThreads = [];
    this.logs = [];
    this.onLog = null;
  }

  init(model) {
    this.model = model;
    const cfg = { M1: [5,1], '1M': [4,4], MM: [6,3] };
    const [uCount, kCount] = cfg[model] || [5,3];
    this.userThreads = Array.from({length: uCount}, (_, i) => new
UserThread(`U${i+1}`));
    this.kernelThreads = Array.from({length: kCount}, (_, i) => new
KernelThread(`K${i+1}`));
  }

  addUserThread() {
    const n = this.userThreads.length + 1;
    const t = new UserThread(`U${n}`);
    this.userThreads.push(t);
    if (this.model === '1M') {
      this.kernelThreads.push(new KernelThread(`K${n}`));
    }
    this.log(`Thread U${n} added to pool`, 'info');
    return t;
  }

  log(msg, cls = 'info') {
    const entry = { msg, cls, time: performance.now() };
    this.logs.unshift(entry);
    if (this.logs.length > 80) this.logs.pop();
    if (this.onLog) this.onLog(msg, cls);
  }

  tick(speed) {
    const m = this.model;
    if (m === 'M1') this._tickManyToOne(speed);
    else if (m === '1M') this._tickOneToOne(speed);
    else this._tickManyToMany(speed);
    return this.userThreads.every(t => t.state === STATE.DONE);
  }

  _tickManyToOne(speed) {
    const kt = this.kernelThreads[0];
    const running = this.userThreads.find(t => t.state === STATE.RUNNING);
    if (!running) {
      const next = this.userThreads.find(t => t.state === STATE.READY);
      if (next) { next.state = STATE.RUNNING; kt.running = next.id; this.log(`K1
dispatches ${next.id}`, 'ok'); }
    } else {
      running.progress = clamp(running.progress + speed * 3, 0, 100);
      if (running.progress >= 100) {
```

```
        this.log(`${running.id} completed (M:1 — sequential)`, 'ok');
        running.state = STATE.DONE; kt.running = null;
      } else if (Math.random() < 0.04) {
        running.state = STATE.WAITING; kt.running = null;
        this.log(`${running.id} I/O block — entire process stalls (M:1
limitation)`, 'warn');
      }
    }
    this.userThreads.filter(t => t.state === STATE.WAITING).forEach(t => {
      if (Math.random() < 0.12) { t.state = STATE.READY; this.log(`${t.id}
unblocked → ready`, 'info'); }
    });
  }

  _tickOneToOne(speed) {
    this.userThreads.forEach((u, i) => {
      const k = this.kernelThreads[i];
      if (!k || u.state === STATE.DONE) return;
      if (u.state === STATE.READY) { u.state = STATE.RUNNING; k.running = u.id;
}
      if (u.state === STATE.RUNNING) {
        u.progress = clamp(u.progress + speed * (1.5 + Math.random() * 2), 0,
100);
        if (u.progress >= 100) {
          u.state = STATE.DONE; k.running = null;
          this.log(`${u.id} done — true parallelism (1:1)`, 'ok');
        } else if (Math.random() < 0.025) {
          u.state = STATE.WAITING; k.running = null;
          this.log(`${u.id} blocks — other threads continue unaffected`,
'warn');
        }
      }
      if (u.state === STATE.WAITING && Math.random() < 0.14) {
        u.state = STATE.READY; this.log(`${u.id} resumed`, 'info');
      }
    });
  }

  _tickManyToMany(speed) {
    this.kernelThreads.forEach(k => {
      if (!k.running) {
        const next = this.userThreads.find(t => t.state === STATE.READY);
        if (next) { next.state = STATE.RUNNING; k.running = next.id;
this.log(`${k.id} multiplexed to ${next.id}`, 'info'); }
      } else {
        const u = this.userThreads.find(t => t.id === k.running);
        if (u) {
          u.progress = clamp(u.progress + speed * (1.2 + Math.random() * 2), 0,
100);
          if (u.progress >= 100) {
            u.state = STATE.DONE; k.running = null;
            this.log(`${u.id} complete on ${k.id}`, 'ok');
          } else if (Math.random() < 0.03) {
            u.state = STATE.WAITING; k.running = null;
            this.log(`${u.id} blocked → ${k.id} picks up next thread`, 'warn');
          }
        }
```

```
      }
    });
    this.userThreads.filter(t => t.state === STATE.WAITING).forEach(t => {
      if (Math.random() < 0.09) { t.state = STATE.READY; this.log(`${t.id} re-
queued`, 'info'); }
    });
  }

  getSnapshot() {
    return {
      userThreads: this.userThreads.map(t => ({...t})),
      kernelThreads: this.kernelThreads.map(k => ({...k})),
      model: this.model
    };
  }
}

// ── Synchronization Engine ──────────────────────────────
class SyncEngine {
  constructor() {
    this.type = 'sem';
    this.threads = [];
    this.semValue = 2;
    this.semMax = 2;
    this.monitorLocked = false;
    this.monitorOwner = null;
    this.bufferSize = 0;
    this.bufferMax = 3;
    this.logs = [];
    this.onLog = null;
  }

  log(msg, cls = 'info') {
    const entry = { msg, cls, time: performance.now() };
    this.logs.unshift(entry);
    if (this.logs.length > 80) this.logs.pop();
    if (this.onLog) this.onLog(msg, cls);
  }

  init(type, opts = {}) {
    this.type = type;
    this.logs = [];
    if (type === 'sem') {
      this.semMax = opts.semMax || 2;
      this.semValue = this.semMax;
      const n = opts.threadCount || 5;
      this.threads = Array.from({length: n}, (_, i) => ({
        id: `T${i+1}`, state: STATE.READY, progress: 0, inCS: false, csTime: 0,
        color: PALETTE[i % PALETTE.length]
      }));
    } else if (type === 'monitor') {
      this.monitorLocked = false; this.monitorOwner = null; this.bufferSize = 0;
      this.threads = [
        {id:'P1', role:'producer', state:STATE.READY, produced:0,
color:PALETTE[0]},
```

```
        {id:'P2', role:'producer', state:STATE.READY, produced:0,
color:PALETTE[1]},
        {id:'C1', role:'consumer', state:STATE.READY, consumed:0,
color:PALETTE[2]},
        {id:'C2', role:'consumer', state:STATE.READY, consumed:0,
color:PALETTE[3]},
      ];
    }
  }

  tick() {
    if (this.type === 'sem') return this._tickSem();
    return this._tickMonitor();
  }

  _tickSem() {
    this.threads.forEach(t => {
      if (t.state === STATE.DONE) return;
      if (t.inCS) {
        t.csTime++; t.progress = clamp(t.progress + 8, 0, 100);
        if (t.csTime >= 4) {
          this.semValue = Math.min(this.semMax, this.semValue + 1);
          t.inCS = false; t.csTime = 0;
          t.state = t.progress >= 100 ? STATE.DONE : STATE.READY;
          if (t.state === STATE.DONE) this.log(`${t.id} completed all
iterations`, 'ok');
          else this.log(`${t.id} exits CS → signal (sem=${this.semValue})`,
'warn');
        }
      } else if (t.state === STATE.READY) {
        if (this.semValue > 0) {
          this.semValue--;
          t.inCS = true; t.state = STATE.RUNNING;
          this.log(`${t.id} wait() → acquired (sem=${this.semValue})`, 'ok');
        } else {
          t.state = STATE.BLOCKED;
          this.log(`${t.id} wait() → blocked (sem=0)`, 'err');
        }
      } else if (t.state === STATE.BLOCKED) {
        if (this.semValue > 0) {
          this.semValue--; t.inCS = true; t.state = STATE.RUNNING;
          this.log(`${t.id} unblocked → enters CS (sem=${this.semValue})`,
'info');
        }
      }
    });
    return this.threads.every(t => t.state === STATE.DONE);
  }

  _tickMonitor() {
    this.threads.forEach(t => {
      if (t.state === STATE.DONE) return;
      if (t.role === 'producer') {
        if (t.state === STATE.READY) {
          if (!this.monitorLocked) {
            this.monitorLocked = true; this.monitorOwner = t.id; t.state =
STATE.RUNNING;
```

```
            this.log(`${t.id} acquires monitor lock`, 'ok');
          } else { t.state = STATE.WAITING; }
        } else if (t.state === STATE.RUNNING) {
          if (this.bufferSize < this.bufferMax) {
            this.bufferSize++;
            t.produced = (t.produced||0) + 1;
            this.log(`${t.id} produced →
buf=${this.bufferSize}/${this.bufferMax}`, 'ok');
            this.monitorLocked = false; this.monitorOwner = null; t.state =
STATE.READY;
            const wc = this.threads.find(x => x.role === 'consumer' && x.state
=== STATE.WAITING);
            if (wc) { wc.state = STATE.READY; this.log(`${t.id} signals ${wc.id}
(notEmpty)`, 'info'); }
          } else {
            this.log(`${t.id} → buffer full, wait(notFull)`, 'warn');
            this.monitorLocked = false; this.monitorOwner = null; t.state =
STATE.WAITING;
          }
        } else if (t.state === STATE.WAITING && !this.monitorLocked) { t.state =
STATE.READY; }
      } else {
        if (t.state === STATE.READY) {
          if (!this.monitorLocked) {
            this.monitorLocked = true; this.monitorOwner = t.id; t.state =
STATE.RUNNING;
            this.log(`${t.id} acquires monitor lock`, 'ok');
          } else { t.state = STATE.WAITING; }
        } else if (t.state === STATE.RUNNING) {
          if (this.bufferSize > 0) {
            this.bufferSize--;
            t.consumed = (t.consumed||0) + 1;
            this.log(`${t.id} consumed ←
buf=${this.bufferSize}/${this.bufferMax}`, 'ok');
            this.monitorLocked = false; this.monitorOwner = null; t.state =
STATE.READY;
            const wp = this.threads.find(x => x.role === 'producer' && x.state
=== STATE.WAITING);
            if (wp) { wp.state = STATE.READY; this.log(`${t.id} signals ${wp.id}
(notFull)`, 'info'); }
          } else {
            this.log(`${t.id} → buffer empty, wait(notEmpty)`, 'warn');
            this.monitorLocked = false; this.monitorOwner = null; t.state =
STATE.WAITING;
          }
        } else if (t.state === STATE.WAITING && !this.monitorLocked) { t.state =
STATE.READY; }
      }
      if
((t.role==='producer'&&t.produced>=6)||(t.role==='consumer'&&t.consumed>=6))
t.state = STATE.DONE;
    });
    return this.threads.every(t => t.state === STATE.DONE);
  }
}

// ── CPU Scheduler Engine ──────────────────────────────────
class SchedulerEngine {
  constructor() {
```

```
    this.processes = [];
    this.algo = 'rr';
    this.quantum = 3;
    this.tick = 0;
    this.currentProc = null;
    this.quantumLeft = 0;
    this.rrQueue = [];
    this.gantt = []; // {id, start, end, color}
    this.ganttCurrent = null;
    this.logs = [];
    this.onLog = null;
  }

  log(msg, cls = 'info') {
    const entry = {msg, cls};
    this.logs.unshift(entry);
    if (this.logs.length > 80) this.logs.pop();
    if (this.onLog) this.onLog(msg, cls);
  }

  init(algo, quantum) {
    this.algo = algo; this.quantum = quantum || 3;
    this.tick = 0; this.currentProc = null; this.quantumLeft = 0;
    this.gantt = []; this.ganttCurrent = null; this.logs = [];
    this.processes = [
      {id:'P1', burst:8, remaining:8, priority:2, arrival:0, state:STATE.READY,
start:-1, finish:-1, wait:0, color:'#3b8cff'},
      {id:'P2', burst:5, remaining:5, priority:1, arrival:1, state:STATE.READY,
start:-1, finish:-1, wait:0, color:'#22c55e'},
      {id:'P3', burst:3, remaining:3, priority:3, arrival:2, state:STATE.READY,
start:-1, finish:-1, wait:0, color:'#f59e0b'},
      {id:'P4', burst:6, remaining:6, priority:2, arrival:0, state:STATE.READY,
start:-1, finish:-1, wait:0, color:'#a855f7'},
    ];
    this.rrQueue = this.processes.map(p => p.id);
  }

  _selectNext() {
    const ready = this.processes.filter(p => p.state === STATE.READY &&
p.arrival <= this.tick);
    if (!ready.length) return null;
    if (this.algo === 'rr') {
      for (const id of this.rrQueue) { const p = ready.find(r => r.id === id);
if (p) return p; }
      return ready[0];
    }
    if (this.algo === 'sjf')      return ready.slice().sort((a,b) => a.remaining
- b.remaining)[0];
    if (this.algo === 'priority') return ready.slice().sort((a,b) => a.priority
- b.priority)[0];
    return ready.slice().sort((a,b) => a.arrival - b.arrival)[0]; // fcfs
  }

  step() {
    this.tick++;
    if (this.currentProc) {
      const p = this.currentProc;
```

```
      p.remaining--;
      this.quantumLeft--;
      // Gantt update
      if (this.ganttCurrent && this.ganttCurrent.id === p.id) {
        this.ganttCurrent.end = this.tick;
      } else {
        if (this.ganttCurrent) this.gantt.push({...this.ganttCurrent});
        this.ganttCurrent = {id: p.id, start: this.tick-1, end: this.tick,
color: p.color};
      }
      if (p.remaining <= 0) {
        p.finish = this.tick; p.state = STATE.DONE;
        this.log(`${p.id} finished at t=${this.tick} (wait=${p.wait})`, 'ok');
        if (this.ganttCurrent) { this.gantt.push({...this.ganttCurrent});
this.ganttCurrent = null; }
        this.currentProc = null; this.quantumLeft = 0;
      } else if (this.algo === 'rr' && this.quantumLeft <= 0) {
        p.state = STATE.READY;
        const idx = this.rrQueue.indexOf(p.id);
        if (idx > -1) { this.rrQueue.splice(idx, 1); this.rrQueue.push(p.id); }
        if (this.ganttCurrent) { this.gantt.push({...this.ganttCurrent});
this.ganttCurrent = null; }
        this.currentProc = null; this.quantumLeft = 0;
        this.log(`${p.id} quantum expired → back to queue`, 'warn');
      }
    }
    if (!this.currentProc) {
      const next = this._selectNext();
      if (next) {
        next.state = STATE.RUNNING;
        if (next.start === -1) next.start = this.tick;
        this.currentProc = next;
        this.quantumLeft = this.quantum;
        this.log(`Scheduler selects ${next.id} (remaining=${next.remaining},
algo=${this.algo.toUpperCase()})`, 'info');
      }
    }
    this.processes.filter(p => p.state === STATE.READY && p.arrival <=
this.tick).forEach(p => p.wait++);
    return this.processes.every(p => p.state === STATE.DONE);
  }


  getStats() {
    const done = this.processes.filter(p => p.state === STATE.DONE);
    const avgWait = done.length ?
(done.reduce((s,p)=>s+p.wait,0)/done.length).toFixed(1) : '-';
    const utilization = this.tick > 0 ?
Math.round(this.gantt.reduce((s,g)=>s+(g.end-g.start),0)/this.tick*100) : 0;
    return { tick: this.tick, done: done.length, total: this.processes.length,
avgWait, utilization };
  }
}


// ── Export engines to global scope ──────────────────────────
window.ThreadingModelEngine = ThreadingModelEngine;
window.SyncEngine = SyncEngine;
window.SchedulerEngine = SchedulerEngine;
```

```
window.STATE = STATE;
window.PALETTE = PALETTE;
```

C.6 File: src/app.js — UI Rendering & Interaction (Key Excerpts)

```
// ============================================================
// MAIN APP — UI Rendering & Interaction
// ============================================================
'use strict';

// ── Helpers ──────────────────────────────────────────────
const $ = id => document.getElementById(id);
const el = (tag, cls, html) => { const e = document.createElement(tag); if(cls)
e.className=cls; if(html!==undefined) e.innerHTML=html; return e; };
const RUNTIME = window.electronAPI?.isElectron ? 'electron' : 'web';
const RUNTIME_LABEL = RUNTIME === 'electron' ? 'Electron Desktop' : 'Web
Browser';
const PLATFORM = window.electronAPI?.platform || 'web';

function chipHTML(id, state, extra = '') {
  const dot = `<span class="state-dot dot-${state}"></span>`;
  return `<div class="thread-row"><span class="thread-chip chip-
${state}">${dot}${id}</span>${extra}</div>`;
}

function logTo(containerId, msg, cls = 'info') {
  const el = $(containerId);
  if (!el) return;
  const t = (performance.now() / 1000).toFixed(2);
  const entry = document.createElement('div');
  entry.className = 'log-entry fade-in';
  entry.innerHTML = `<span class="log-time">[${t}s]</span><span class="log-
${cls}">${msg}</span>`;
  el.prepend(entry);
  while (el.children.length > 60) el.removeChild(el.lastChild);
}

function clearLog(id) { const e = $(id); if(e) e.innerHTML = ''; }

// ── Build App Shell ──────────────────────────────────────
function buildShell() {
  document.body.dataset.runtime = RUNTIME;
  document.body.dataset.platform = PLATFORM;
  document.getElementById('app').innerHTML = `
    <div class="titlebar">
      <div class="titlebar-logo">⬡</div>
      <span class="titlebar-title">ThreadLab</span>
      <span class="titlebar-subtitle">multithreading simulator</span>
      <div class="titlebar-spacer"></div>
      <span class="titlebar-runtime ${RUNTIME}">${RUNTIME_LABEL}</span>
      <span class="titlebar-badge" id="tick-badge">tick: 0</span>
    </div>
```

```
    <div class="tabs-bar">
      <button class="tab-btn active" data-tab="model"><span class="tab-
icon">🧵</span>Threading Models</button>
      <button class="tab-btn" data-tab="sync"><span class="tab-
icon">⚙️</span>Synchronization</button>
      <button class="tab-btn" data-tab="sched"><span class="tab-
icon">⏱️</span>CPU Scheduler</button>
    </div>

    <div class="content">
      <div class="panel active" id="panel-model"></div>
      <div class="panel" id="panel-sync"></div>
      <div class="panel" id="panel-sched"></div>
    </div>`;

  document.querySelectorAll('.tab-btn').forEach(btn => {
    btn.addEventListener('click', () => {
      document.querySelectorAll('.tab-btn').forEach(b =>
b.classList.remove('active'));
      document.querySelectorAll('.panel').forEach(p =>
p.classList.remove('active'));
      btn.classList.add('active');
      $(`panel-${btn.dataset.tab}`).classList.add('active');
    });
  });
}

// ═══════════════════════════════════════════════════════════════
// TAB 1 — THREADING MODELS
// ═══════════════════════════════════════════════════════════════
let modelEngine = new ThreadingModelEngine();
let modelInterval = null;
let modelRunning = false;
let modelTick = 0;

const MODEL_DESCS = {
  M1: `<b>Many-to-One (M:1):</b> All user threads share a single kernel thread.
Only one runs at a time — no true parallelism. A single I/O block stalls the
entire process. Simple to implement but limited scalability. Used in early
green-thread implementations.`,
  '1M': `<b>One-to-One (1:1):</b> Each user thread maps to its own kernel
thread. True parallelism on multi-core CPUs. One blocking thread does not affect
others. Higher overhead for thread creation. Used by Linux pthreads, Windows
threads.`,
  MM: `<b>Many-to-Many (M:N):</b> M user threads multiplexed over N kernel
threads (N ≤ M). Best of both worlds — efficient creation, true parallelism, and
blocking one kernel thread doesn't stall all user threads. Used by Go
goroutines, Java virtual threads (JEP 444).`
};

function buildModelTab() {
  $('panel-model').innerHTML = `
    <div class="ctrl-row">
      <span class="ctrl-label">Model:</span>
      <select class="ctrl-sel" id="model-sel">
        <option value="M1">Many-to-One (M:1)</option>
```

```
        <option value="1M">One-to-One (1:1)</option>
        <option value="MM" selected>Many-to-Many (M:N)</option>
      </select>
      <button class="btn primary" id="btn-model-run">▶️ Run</button>
      <button class="btn" id="btn-model-stop">■ Stop</button>
      <button class="btn" id="btn-model-add">+ Thread</button>
      <span class="ctrl-label" style="margin-left:8px">Speed:</span>
      <div class="speed-wrap">
        <input type="range" class="speed-slider" id="speed-model" min="1"
max="10" value="5">
        <span class="ctrl-label" id="speed-model-lbl">5x</span>
      </div>
    </div>
    <div class="panel-scroll">
      <div class="grid-2" style="margin-bottom:12px">
        <div class="card">
          <div class="card-title">User Threads</div>
          <div id="model-uthreads"></div>
        </div>
        <div class="card">
          <div class="card-title">Kernel Threads</div>
          <div class="cpu-grid" id="model-kthreads"></div>
        </div>
      </div>
      <div class="card" style="margin-bottom:12px">
        <div class="card-title">Thread → Kernel Mapping</div>
        <div class="map-canvas" id="map-canvas"><svg id="map-svg" viewBox="0 0
600 180"></svg></div>
      </div>
      <div class="grid-2" style="margin-bottom:12px">
        <div class="card">
          <div class="card-title">Model Description</div>
          <div class="model-desc-box" id="model-desc"></div>
        </div>
        <div class="card">
          <div class="card-title">Activity Log</div>
          <div class="log-panel" id="log-model"></div>
        </div>
      </div>
    </div>`;

  $('model-sel').addEventListener('change', () => { stopModel(); initModel();
renderModel(); });
  $('speed-model').addEventListener('input', e => { $('speed-model-
lbl').textContent = e.target.value + 'x'; });
  $('btn-model-run').addEventListener('click', startModel);
  $('btn-model-stop').addEventListener('click', stopModel);
  $('btn-model-add').addEventListener('click', () => {
    modelEngine.addUserThread();
    renderModel();
  });
  initModel();
  renderModel();
}

function initModel() {
  const model = $('model-sel').value;
```

```
  modelEngine = new ThreadingModelEngine();
  modelEngine.onLog = (msg, cls) => logTo('log-model', msg, cls);
  modelEngine.init(model);
  $('model-desc').innerHTML = MODEL_DESCS[model] || '';
  modelTick = 0;
}

function startModel() {
  if (modelRunning) return;
  stopModel(); initModel(); clearLog('log-model');
  modelRunning = true;
  $('btn-model-run').disabled = true;
  const tick = () => {
    const speed = parseInt($('speed-model').value) || 5;
    modelTick++;
    $('tick-badge').textContent = `tick: ${modelTick}`;
    const done = modelEngine.tick(speed);
    renderModel();
    if (done) { stopModel(); logTo('log-model', 'All threads completed ✓',
'ok'); }
  };
  modelInterval = setInterval(tick, 280);
}

function stopModel() {
  modelRunning = false;
  if (modelInterval) { clearInterval(modelInterval); modelInterval = null; }
  if ($('btn-model-run')) $('btn-model-run').disabled = false;
}

function renderModel() {
  const snap = modelEngine.getSnapshot();
  // User threads
  $('model-uthreads').innerHTML = snap.userThreads.map(t => {
    const pct = Math.round(t.progress);
    const bar = `<div style="display:flex;align-items:center;gap:6px;flex:1">
      <div class="prog-bar-bg"><div class="prog-bar-fill"
style="width:${pct}%;background:${t.color}"></div></div>
      <span class="thread-meta">${pct}%</span></div>`;
    return chipHTML(t.id, t.state === STATE.RUNNING ? 'running' : t.state, bar);
  }).join('');

  // Kernel threads
  $('model-kthreads').innerHTML = snap.kernelThreads.map(k => {
    const busy = k.running !== null;
    const ut = busy ? snap.userThreads.find(u => u.id === k.running) : null;
    const col = ut ? ut.color : '';
    return `<div class="cpu-core ${busy ? 'busy' : ''}">
      <div class="core-label">${k.id}</div>
      ${busy ? `<div class="core-thread"
style="color:${col}">${k.running}</div>` : '<div class="core-idle">idle</div>'}
    </div>`;
  }).join('');

  drawMappingSVG(snap);
}
```

```
function drawMappingSVG(snap) {
  const svg = $('map-svg');
  const uCount = snap.userThreads.length;
  const kCount = snap.kernelThreads.length;
  const m = snap.model;
  const uSpacing = 560 / (uCount + 1);
  const kSpacing = 560 / (kCount + 1);
  const uy = 25, ky = 110, cpu_y = 162;

  let d = `<defs><marker id="ma" viewBox="0 0 10 10" refX="8" refY="5"
markerWidth="5" markerHeight="5" orient="auto"><path d="M2 2L8 5L2 8"
fill="none" stroke="#3b8cff" stroke-width="1.5"/></marker></defs>`;

  // CPU bar
  d += `<rect x="80" y="${cpu_y}" width="440" height="14" rx="4" fill="#1e2636"
stroke="#2a3347" stroke-width="0.5"/>`;
  d += `<text x="300" y="${cpu_y+10}" text-anchor="middle" font-size="9"
fill="#4a5568" font-family="JetBrains Mono">CPU HARDWARE</text>`;

  // Kernel threads
  snap.kernelThreads.forEach((k, i) => {
    const kx = kSpacing * (i + 1) + 20;
    const busy = k.running !== null;
    const ut = busy ? snap.userThreads.find(u => u.id === k.running) : null;
    const fillCol = busy ? '#0d2818' : '#1e2636';
    const strokeCol = busy ? '#165a2e' : '#2a3347';
    const textCol = busy && ut ? ut.color : '#6b7a94';
    d += `<rect x="${kx-22}" y="${ky}" width="44" height="22" rx="5"
fill="${fillCol}" stroke="${strokeCol}" stroke-width="0.5"/>`;
    d += `<text x="${kx}" y="${ky+15}" text-anchor="middle" font-size="10"
fill="${textCol}" font-family="JetBrains Mono" font-
weight="500">${k.id}</text>`;
    d += `<line x1="${kx}" y1="${ky+22}" x2="${kx}" y2="${cpu_y}"
stroke="#2a3347" stroke-width="0.5" stroke-dasharray="2 2"/>`;
  });

  // User threads + mapping lines
  snap.userThreads.forEach((u, i) => {
    const ux = uSpacing * (i + 1) + 20;
    let kx;
    if (m === 'M1') kx = kSpacing + 20;
    else if (m === '1M') kx = kSpacing * (i + 1) + 20;
    else kx = kSpacing * ((i % snap.kernelThreads.length) + 1) + 20;

    const running = u.state === 'running';
    const alpha = running ? '1' : '0.25';
    d += `<line x1="${ux}" y1="${uy+18}" x2="${kx}" y2="${ky}"
stroke="${u.color}" stroke-width="${running?1.5:0.5}" opacity="${alpha}" stroke-
dasharray="${running?'none':'3 2'}"/>`;

    const light =
{ready:'#0d1e33',running:'#0d2818',waiting:'#271d08',blocked:'#250d0d',done:'#1a
1f2a'}[u.state]||'#1e2636';
    const border =
{ready:'#1a4a7a',running:'#165a2e',waiting:'#5a4210',blocked:'#5a1a1a',done:'#2a
3347'}[u.state]||'#2a3347';
```

```
    d += `<rect x="${ux-17}" y="${uy}" width="34" height="18" rx="4"
fill="${light}" stroke="${border}" stroke-width="0.5"/>`;
    d += `<text x="${ux}" y="${uy+13}" text-anchor="middle" font-size="9"
fill="${u.color}" font-family="JetBrains Mono" font-
weight="500">${u.id}</text>`;
  });

  svg.innerHTML = d;
}

// ═══════════════════════════════════════════════════════════════
// TAB 2 — SYNCHRONIZATION
// ═══════════════════════════════════════════════════════════════
let syncEngine = new SyncEngine();
let syncInterval = null;
let syncRunning = false;

function buildSyncTab() {
  $('panel-sync').innerHTML = `
    <div class="ctrl-row">
      <span class="ctrl-label">Primitive:</span>
      <select class="ctrl-sel" id="sync-sel">
        <option value="sem">Semaphore</option>
        <option value="monitor">Monitor (Mutex + CV)</option>
      </select>
      <div id="sync-extra-ctrl" style="display:flex;gap:8px;align-
items:center"></div>
      <button class="btn primary" id="btn-sync-run">▶️ Run</button>
      <button class="btn" id="btn-sync-stop">■ Stop</button>
    </div>
    <div class="panel-scroll">
      <div class="grid-2" style="margin-bottom:12px">
        <div class="card" id="sync-state-card">
          <div class="card-title" id="sync-state-title">Primitive State</div>
          <div id="sync-state-body"></div>
        </div>
        <div class="card">
          <div class="card-title">Thread States</div>
          <div id="sync-threads"></div>
        </div>
      </div>
      <div class="card" style="margin-bottom:12px">
        <div class="card-title">Visualization</div>
        <div class="sync-canvas" id="sync-canvas"><svg id="sync-svg" viewBox="0
0 600 150"></svg></div>
      </div>
      <div class="card">
        <div class="card-title">Event Log</div>
        <div class="log-panel" id="log-sync"></div>
      </div>
    </div>`;

  $('sync-sel').addEventListener('change', () => { stopSync();
buildSyncExtraCtrl(); initSync(); renderSync(); });
  $('btn-sync-run').addEventListener('click', startSync);
  $('btn-sync-stop').addEventListener('click', stopSync);
```

```
  buildSyncExtraCtrl();
  initSync();
  renderSync();
}

function buildSyncExtraCtrl() {
  const type = $('sync-sel').value;
  const extra = $('sync-extra-ctrl');
  if (type === 'sem') {
    extra.innerHTML = `<span class="ctrl-label">Initial value:</span>
      <input type="number" class="ctrl-num" id="sem-init" min="1" max="5"
value="2" style="width:52px">
      <span class="ctrl-label">Threads:</span>
      <input type="number" class="ctrl-num" id="sync-tcount" min="2" max="8"
value="5" style="width:52px">`;
  } else {
    extra.innerHTML = `<span class="ctrl-label">Producer×2 · Consumer×2 ·
Buffer=3</span>`;
  }
}

function initSync() {
  const type = $('sync-sel').value;
  syncEngine = new SyncEngine();
  syncEngine.onLog = (msg, cls) => logTo('log-sync', msg, cls);
  const opts = {};
  if (type === 'sem') {
    opts.semMax = parseInt($('sem-init')?.value) || 2;
    opts.threadCount = parseInt($('sync-tcount')?.value) || 5;
  }
  syncEngine.init(type, opts);
  $('sync-state-title').textContent = {sem:'Semaphore State',monitor:'Monitor /
Buffer State'}[type];
}

function startSync() {
  if (syncRunning) return;
  stopSync(); clearLog('log-sync'); initSync(); syncRunning = true;
  $('btn-sync-run').disabled = true;
  syncInterval = setInterval(() => {
    const done = syncEngine.tick();
    renderSync();
    if (done) { stopSync(); logTo('log-sync','Simulation complete ✓','ok'); }
  }, 480);
}

function stopSync() {
  syncRunning = false;
  if (syncInterval) { clearInterval(syncInterval); syncInterval = null; }
  if ($('btn-sync-run')) $('btn-sync-run').disabled = false;
}

function renderSync() {
  const type = $('sync-sel').value;
  const threads = syncEngine.threads;
```

```
  // Thread states
  $('sync-threads').innerHTML = threads.map(t => {
    let meta = '';
    if (type === 'monitor' && t.role) meta = `<span class="thread-
meta">${t.role}${t.produced!=null?' prod='+t.produced:''}${t.consumed!=null?'
cons='+t.consumed:''}</span>`;
    if (type === 'sem' && t.inCS) meta = `<span class="thread-meta"
style="color:#3b8cff">in CS</span>`;
    const st = t.state === STATE.RUNNING ? 'running' : t.state;
    return chipHTML(t.id, st, meta);
  }).join('');

  // State panel
  const sb = $('sync-state-body');
  if (type === 'sem') {
    const v = syncEngine.semValue, max = syncEngine.semMax;
    const cls = v === 0 ? 'zero' : v === max ? 'full' : '';
    const blocked = threads.filter(t => t.state === STATE.BLOCKED).map(t =>
t.id);
    const inCS = threads.filter(t => t.inCS).map(t => t.id);
    sb.innerHTML = `
      <div class="sem-block">
        <div class="sem-counter ${cls}">${v}</div>
        <div>
          <div class="sem-info">Value: ${v} / ${max} ${v===0?'(all slots
taken)':'('+v+' available)'}</div>
          <div class="sem-info-sub">Blocked: ${blocked.join(', ')||'none'}</div>
          <div class="sem-info-sub">In critical section: ${inCS.join(',
')||'none'}</div>
        </div>
      </div>
      <div style="display:flex;gap:4px;margin-top:6px">${
        Array.from({length:max},(_,i)=>`<div
style="width:20px;height:8px;border-
radius:2px;background:${i<v?'#22c55e':'#250d0d'};border:1px solid
${i<v?'#165a2e':'#5a1a1a'}"></div>`).join('')
      }</div>`;
  } else if (type === 'monitor') {
    const bs = syncEngine.bufferSize, bm = syncEngine.bufferMax;
    sb.innerHTML = `
      <div style="font-size:11px;line-height:1.9;font-family:var(--mono)">
        <div>Lock: <b
style="color:${syncEngine.monitorLocked?'#f59e0b':'#22c55e'}">${syncEngine.monit
orLocked?'🔒 '+syncEngine.monitorOwner:'🔓 free'}</b></div>
        <div>Buffer: <span style="font-family:var(--mono);letter-
spacing:2px">${Array.from({length:bm},(_,i)=>i<bs?'▮':'▯').join(' ')}</span>
${bs}/${bm}</div>
        <div>Waiting:
${threads.filter(t=>t.state===STATE.WAITING).map(t=>t.id).join(',
')||'none'}</div>
      </div>`;
  }

  drawSyncSVG(type);
}

function drawSyncSVG(type) {
```

```
  const svg = $('sync-svg');
  const threads = syncEngine.threads;
  let d = `<defs><marker id="sa" viewBox="0 0 10 10" refX="8" refY="5"
markerWidth="5" markerHeight="5" orient="auto"><path d="M2 2L8 5L2 8"
fill="none" stroke="#3d4a60" stroke-width="1.5"/></marker></defs>`;


  if (type === 'sem') {
    const n = threads.length, sp = 560 / (n + 1);
    threads.forEach((t, i) => {
      const x = sp*(i+1)+20, y=20;
      const fills =
{ready:'#0d1e33',running:'#0d2818',blocked:'#250d0d',done:'#1a1f2a',waiting:'#27
1d08'};
      const strokes =
{ready:'#1a4a7a',running:'#165a2e',blocked:'#5a1a1a',done:'#2a3347',waiting:'#5a
4210'};
      const txtcols =
{ready:'#3b8cff',running:'#22c55e',blocked:'#ef4444',done:'#6b7a94',waiting:'#f5
9e0b'};
      const st = t.state===STATE.RUNNING?'running':t.state;
      d += `<rect x="${x-18}" y="${y}" width="36" height="20" rx="4"
fill="${fills[st]||'#1a1f2a'}" stroke="${strokes[st]||'#2a3347'}" stroke-
width="0.5"/>`;
      d += `<text x="${x}" y="${y+14}" text-anchor="middle" font-size="10"
fill="${txtcols[st]||'#6b7a94'}" font-family="JetBrains Mono" font-
weight="500">${t.id}</text>`;
      if (t.inCS) d += `<line x1="${x}" y1="${y+20}" x2="300" y2="80"
stroke="${t.color||'#22c55e'}" stroke-width="1.5"/>`;
      else if (t.state===STATE.BLOCKED) d += `<line x1="${x}" y1="${y+20}"
x2="300" y2="80" stroke="#ef4444" stroke-width="0.5" stroke-dasharray="3 2"/>`;
    });
    const v = syncEngine.semValue, max = syncEngine.semMax;
    const cls_c = v===0?'#250d0d':v===max?'#0d2818':'#1e2636';
    const cls_s = v===0?'#5a1a1a':v===max?'#165a2e':'#2a3347';
    const cls_t = v===0?'#ef4444':v===max?'#22c55e':'#b0bac8';
    d += `<rect x="264" y="68" width="72" height="44" rx="8" fill="${cls_c}"
stroke="${cls_s}" stroke-width="1"/>`;
    d += `<text x="300" y="87" text-anchor="middle" font-size="9"
fill="${cls_t}" font-family="JetBrains Mono">SEMAPHORE</text>`;
    d += `<text x="300" y="103" text-anchor="middle" font-size="20"
fill="${cls_t}" font-family="JetBrains Mono" font-weight="600">${v}</text>`;
    d += `<text x="300" y="130" text-anchor="middle" font-size="9"
fill="#3d4a60" font-family="JetBrains Mono">critical section</text>`;
  } else if (type === 'monitor') {
    // Buffer cells
    const bm = syncEngine.bufferMax;
    for (let i=0;i<bm;i++) {
      const x = 200+i*70, filled = i<syncEngine.bufferSize;
      d += `<rect x="${x}" y="65" width="54" height="36" rx="5"
fill="${filled?'#0d2818':'#1e2636'}" stroke="${filled?'#165a2e':'#2a3347'}"
stroke-width="0.5"/>`;
      if (filled) d += `<text x="${x+27}" y="${65+24}" text-anchor="middle"
font-size="16" fill="#22c55e" font-family="JetBrains Mono">■</text>`;
    }
    d += `<text x="305" y="120" text-anchor="middle" font-size="9"
fill="#4a5568" font-family="JetBrains Mono">BUFFER
(${syncEngine.bufferSize}/${bm})</text>`;
    // Lock
    const lk = syncEngine.monitorLocked;
```

```
    d += `<rect x="254" y="30" width="100" height="26" rx="4"
fill="${lk?'#271d08':'#1e2636'}" stroke="${lk?'#5a4210':'#2a3347'}" stroke-
width="0.5"/>`;
    d += `<text x="304" y="47" text-anchor="middle" font-size="10"
fill="${lk?'#f59e0b':'#22c55e'}" font-family="JetBrains Mono">${lk?'🔒
'+syncEngine.monitorOwner:'🔓 free'}</text>`;
    // Producers left, consumers right
    threads.forEach((t,i) => {
      const st = t.state===STATE.RUNNING?'running':t.state;
      const fills =
{ready:'#0d1e33',running:'#0d2818',waiting:'#271d08',done:'#1a1f2a'};
      const strokes =
{ready:'#1a4a7a',running:'#165a2e',waiting:'#5a4210',done:'#2a3347'};
      const txtcols =
{ready:'#3b8cff',running:'#22c55e',waiting:'#f59e0b',done:'#6b7a94'};
      if (t.role==='producer') {
        const x=30+i*70, y=65;
        d += `<rect x="${x}" y="${y}" width="54" height="22" rx="4"
fill="${fills[st]||'#1e2636'}" stroke="${strokes[st]||'#2a3347'}" stroke-
width="0.5"/>`;
        d += `<text x="${x+27}" y="${y+15}" text-anchor="middle" font-size="10"
fill="${txtcols[st]||'#6b7a94'}" font-family="JetBrains Mono" font-
weight="500">${t.id}</text>`;
        d += `<line x1="${x+54}" y1="${y+11}" x2="198" y2="83" stroke="#2a3347"
stroke-width="0.5" marker-end="url(#sa)"/>`;
      } else {
        const ci = threads.filter(x=>x.role==='consumer').indexOf(t);
        const x=462+ci*70, y=65;
        d += `<rect x="${x}" y="${y}" width="54" height="22" rx="4"
fill="${fills[st]||'#1e2636'}" stroke="${strokes[st]||'#2a3347'}" stroke-
width="0.5"/>`;
        d += `<text x="${x+27}" y="${y+15}" text-anchor="middle" font-size="10"
fill="${txtcols[st]||'#6b7a94'}" font-family="JetBrains Mono" font-
weight="500">${t.id}</text>`;
        d += `<line x1="${x}" y1="${y+11}" x2="${200+bm*70}" y2="83"
stroke="#2a3347" stroke-width="0.5" marker-end="url(#sa)"/>`;
      }
    });
  }

  svg.innerHTML = d;
}

// ═══════════════════════════════════════════════════════════
let schedEngine = new SchedulerEngine();
let schedInterval = null;
let schedRunning = false;

function buildSchedTab() {
  $('panel-sched').innerHTML = `
    <div class="ctrl-row">
      <span class="ctrl-label">Algorithm:</span>
      <select class="ctrl-sel" id="sched-algo">
        <option value="rr">Round Robin</option>
        <option value="sjf">Shortest Job First</option>
        <option value="priority">Priority Scheduling</option>
        <option value="fcfs">FCFS</option>
      </select>
```

```
      <span class="ctrl-label">Quantum:</span>
      <input type="number" class="ctrl-num" id="quantum-val" min="1" max="10"
value="3">
      <button class="btn primary" id="btn-sched-run">▶️ Run</button>
      <button class="btn" id="btn-sched-stop">■ Stop</button>
      <button class="btn" id="btn-sched-reset">↺ Reset</button>
    </div>
    <div class="panel-scroll">
      <div class="stats-row" id="sched-stats"></div>
      <div class="grid-2" style="margin-bottom:12px">
        <div class="card">
          <div class="card-title">Ready Queue</div>
          <div id="ready-queue" style="display:flex;gap:6px;flex-wrap:wrap;min-
height:32px;align-items:center;padding:2px 0"></div>
        </div>
        <div class="card">
          <div class="card-title">CPU Core</div>
          <div id="sched-cpu"></div>
        </div>
      </div>
      <div class="card" style="margin-bottom:12px">
        <div class="card-title">Gantt Chart</div>
        <div class="gantt-wrap" id="gantt-area"></div>
      </div>
      <div class="card">
        <div class="card-title">Process Table</div>
        <div id="proc-table" style="overflow-x:auto"></div>
      </div>
    </div>`;

  $('btn-sched-run').addEventListener('click', startSched);
  $('btn-sched-stop').addEventListener('click', stopSched);
  $('btn-sched-reset').addEventListener('click', resetSched);
  resetSched();
}

function resetSched() {
  stopSched();
  schedEngine = new SchedulerEngine();
  schedEngine.init($('sched-algo')?.value||'rr', parseInt($('quantum-
val')?.value)||3);
  renderSched();
}

function startSched() {
  if (schedRunning) return;
  stopSched();
  schedEngine = new SchedulerEngine();
  schedEngine.init($('sched-algo').value, parseInt($('quantum-val').value)||3);
  schedRunning = true;
  $('btn-sched-run').disabled = true;
  schedInterval = setInterval(() => {
    const done = schedEngine.step();
    renderSched();
    if (done || schedEngine.tick > 80) { stopSched(); }
  }, 350);
```

```
}

function stopSched() {
  schedRunning = false;
  if (schedInterval) { clearInterval(schedInterval); schedInterval = null; }
  if ($('btn-sched-run')) $('btn-sched-run').disabled = false;
}

function renderSched() {
  const procs = schedEngine.processes;
  const stats = schedEngine.getStats();
  const maxTick = Math.max(schedEngine.tick, 1);

  // Stats
  $('sched-stats').innerHTML = `
    <div class="stat-chip"><span class="stat-val">${stats.tick}</span><span
class="stat-lbl">Tick</span></div>
    <div class="stat-chip"><span class="stat-
val">${stats.done}/${stats.total}</span><span class="stat-lbl">Done</span></div>
    <div class="stat-chip"><span class="stat-val">${stats.avgWait}</span><span
class="stat-lbl">Avg Wait</span></div>
    <div class="stat-chip"><span class="stat-
val">${stats.utilization}%</span><span class="stat-lbl">CPU Util.</span></div>`;

  // Ready queue
  const rq = $('ready-queue');
  const ready = procs.filter(p => p.state === STATE.READY && p.arrival <=
schedEngine.tick);
  if (ready.length) {
    rq.innerHTML = ready.map((p,i) =>
      `${i>0?'<span class="rq-arrow">›</span>':''}<span class="rq-chip"
style="border-
color:${p.color};color:${p.color};background:${p.color}18">${p.id}<span
style="opacity:.6;font-size:9px;margin-left:3px">${p.remaining}</span></span>`
    ).join('');
  } else {
    rq.innerHTML = `<span style="font-size:11px;color:var(--
text3)">empty</span>`;
  }

  // CPU
  const cpu = $('sched-cpu');
  const cur = schedEngine.currentProc;
  if (cur) {
    const pct = Math.round((1 - cur.remaining/cur.burst)*100);
    cpu.innerHTML = `<div style="display:flex;align-
items:center;gap:12px;padding:6px 0">
      <div style="width:44px;height:44px;border-
radius:50%;background:${cur.color}18;border:2px solid
${cur.color};display:flex;align-items:center;justify-content:center;font-
size:13px;font-weight:600;color:${cur.color};font-family:var(--
mono)">${cur.id}</div>
      <div style="flex:1">
        <div style="font-size:11px;color:var(--text2);margin-
bottom:6px">${cur.id} — ${pct}% complete · remaining: ${cur.remaining} ticks ·
quantum left: ${schedEngine.quantumLeft}</div>
        <div class="prog-bar-bg" style="height:8px"><div class="prog-bar-fill"
style="width:${pct}%;background:${cur.color};height:8px"></div></div>
```

```
      </div>
    </div>`;
  } else {
    cpu.innerHTML = `<div style="font-size:11px;color:var(--text3);padding:12px
0">— idle —</div>`;
  }

  // Gantt
  const allSegs = [...schedEngine.gantt];
  if (schedEngine.ganttCurrent) allSegs.push(schedEngine.ganttCurrent);
  const ganttEl = $('gantt-area');
  ganttEl.innerHTML = procs.map(p => {
    const segs = allSegs.filter(s => s.id === p.id);
    const bars = segs.map(s => {
      const left = (s.start / maxTick * 100).toFixed(2);
      const width = Math.max(((s.end - s.start) / maxTick * 100),
0.3).toFixed(2);
      return `<div class="gantt-seg"
style="left:${left}%;width:${width}%;background:${p.color}33;border-
color:${p.color};color:${p.color}"></div>`;
    }).join('');
    return `<div class="gantt-row-wrap">
      <div class="gantt-pid">${p.id}</div>
      <div class="gantt-track">${bars}</div>
    </div>`;
  }).join('');

  // Process table
  $('proc-table').innerHTML = `<table style="width:100%;border-
collapse:collapse;font-size:11px;font-family:var(--mono)">
    <tr style="border-bottom:1px solid var(--
border)">${['PID','Burst','Remaining','Priority','Arrival','State','Wait'].map(h
=>`<th style="padding:5px 8px;text-align:left;font-size:9px;font-
weight:600;color:var(--text2);text-transform:uppercase;letter-
spacing:.05em">${h}</th>`).join('')}</tr>
    ${procs.map(p=>`<tr style="border-bottom:1px solid var(--border)">
      <td style="padding:4px 8px;font-weight:600;color:${p.color}">${p.id}</td>
      <td style="padding:4px 8px;color:var(--text1)">${p.burst}</td>
      <td style="padding:4px 8px;color:var(--text1)">${p.remaining}</td>
      <td style="padding:4px 8px;color:var(--text1)">${p.priority}</td>
      <td style="padding:4px 8px;color:var(--text1)">${p.arrival}</td>
      <td style="padding:4px 8px"><span class="thread-chip chip-
${p.state===STATE.DONE?'done':p.state===STATE.RUNNING?'running':'ready'}"
style="font-size:9px;padding:2px 7px">${p.state}</span></td>
      <td style="padding:4px 8px;color:var(--text1)">${p.wait}</td>
    </tr>`).join('')}
  </table>`;
}

// ── Init ──────────────────────────────────────────────────
buildShell();
buildModelTab();
buildSyncTab();
buildSchedTab();
```

8. Links

GitHub Repository Link: *https://github.com/deepak179-s/ThreadLab*

LinkedIn Post Link:
https://www.linkedin.com/feed/update/urn:li:ugcPost:7454568081663791104/

Youtube Link: *https://www.youtube.com/watch?v=-rYxoxJ96Dc*